

PlacementPath – AWS-Based
Full Stack Placement Preparation Portal

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Bethala Sumana, APSSDC

Mr. Juttuka Lovababu, APSSDC

submitted by

Kosanam Balagangadhar, 25BQ5A4706, VVIT Nambur (Batch -2)

Dupam Naga Santhosh, 25BQ5A4703, VVIT Nambur (Batch -2)

Annavarapu Soma Sekhar, 24BQ1A4704, VVIT Nambur (Batch -2)

Puli Jagadeesh, 25BQ5A4711, VVIT Nambur (Batch -2)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh State Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs.Sumana Bethala and Mr. Juttuka Lovababu, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh State Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank You.

Sincerely,

Kosanam Balagangadhar, 25BQ5A4706, balagangadharkosanam99@gmail.com

Dupam Naga Santhosh, 25BQ5A4703, santhoshdupam2313@gmail.com

Annavarapu Soma Sekhar, 24BQ1A4704, somasekharannavarapu@gmail.com

Puli Jagadeesh, 25BQ5A4711, jagadeesh20052006@gmail.com

ABSTRACT

PlacementPath is a cloud-based full stack web application developed to help students prepare for campus placements through a single online platform. The project was developed as part of the **Summer Online Internship – AWS Cloud Computing** to demonstrate the practical use of AWS services along with modern web development technologies. The application provides a simple and secure way for students to register, log in, and access placement-related resources and company information.

The application follows a **three-tier architecture**. The frontend was developed using HTML, CSS, and JavaScript to create a responsive and user-friendly interface. The backend was developed using Python and the Flask framework, which processes user requests and manages the application logic. User authentication is implemented using JSON Web Tokens (JWT), and passwords are securely stored using password hashing techniques.

The project was deployed on **Amazon Web Services (AWS)** using five major services. The frontend is hosted on **Amazon S3**, while the backend runs on an **Amazon EC2** instance with Nginx and Gunicorn. The application data is stored in **Amazon RDS** using the **MySQL** database engine. **Amazon CloudFront** is configured with separate origins for Amazon S3 and Amazon EC2 to deliver both static website files and backend API requests efficiently. **AWS IAM** is used to manage secure access to AWS resources.

Overall, PlacementPath demonstrates how a full stack web application can be successfully deployed on the AWS cloud using industry-standard services. The project provides practical experience in cloud deployment, secure authentication, database management, and web application hosting while offering a centralized platform for placement preparation.

The project also improves understanding of cloud infrastructure, web hosting, and service integration in a real-world environment. It demonstrates how different AWS services can work together to build a reliable and efficient application. This project serves as a practical example of deploying a secure full stack application on the cloud and provides valuable hands-on experience in AWS Cloud Computing, making it a useful learning resource for students and beginners.

TABLE OF CONTENTS

S.no	Content	Pg.No
1.	INTRODUCTION.....	1
2.	METHODOLOGY.....	2
3.	TECHNOLOGY STACK.....	5
4.	SYSTEM DESIGN / ARCHITECTURE.....	8
5.	IMPLEMENTATION.....	19
6.	RESULTS.....	22
7.	CONCLUSION.....	27

1. INTRODUCTION

In today's digital world, cloud computing has become one of the most important technologies for developing and deploying modern web applications. Organizations increasingly rely on cloud platforms to build scalable, secure, and highly available applications while reducing infrastructure management and operational costs. Amazon Web Services (AWS), one of the leading cloud service providers, offers a wide range of services that enable developers to design, deploy, and manage applications efficiently. This project demonstrates the practical implementation of cloud computing concepts by deploying a full stack web application using multiple AWS services.

Placement preparation is an important phase in every student's academic journey. Students often search for learning resources, company information, interview experiences, aptitude materials, and preparation roadmaps from different websites and platforms. Since this information is scattered across multiple sources, students spend considerable time searching instead of preparing effectively. The absence of a centralized platform makes placement preparation more difficult and less organized.

To address this problem, *PlacementPath* was developed as a cloud-based full stack web application that provides a centralized platform for placement preparation. The application allows students to register, log in securely, and access placement-related resources, company information, and structured preparation roadmaps through a simple and user-friendly interface. By bringing all essential placement resources together in a single application, the system helps students prepare more efficiently and improves accessibility to important learning materials.

The application follows a **three-tier architecture**, which separates the presentation layer, application layer, and data layer. The frontend was developed using HTML, CSS, and JavaScript to provide a responsive and interactive user interface. The backend was implemented using Python and the Flask framework to process user requests, handle authentication, and communicate with the database. Secure authentication is achieved using JSON Web Tokens (JWT), while password hashing techniques are used to protect user credentials and improve application security.

One of the primary objectives of this project is to demonstrate the practical use of **Amazon Web Services (AWS)** for deploying a real-world full stack application. The

frontend is hosted on **Amazon S3** as a static website, while the backend Flask application is deployed on an **Amazon EC2** instance configured with **Nginx** as the reverse proxy and **Gunicorn** as the WSGI application server. Application data is stored in **Amazon RDS** using the MySQL database engine. **Amazon CloudFront** is configured with two origins—Amazon S3 for static website content and Amazon EC2 for backend API requests—allowing efficient delivery of both static and dynamic content through a single CloudFront distribution. **AWS Identity and Access Management (IAM)** is used to securely manage user permissions and access to AWS resources.

The project demonstrates how multiple AWS services can be integrated to build a secure, scalable, and reliable cloud-based application. It also provides practical experience in cloud deployment, web hosting, database management, authentication, content delivery, and server configuration. Through this project, important concepts such as three-tier architecture, cloud infrastructure, secure application deployment, and service integration have been successfully implemented using industry-standard AWS technologies.

This report presents the complete development process of *PlacementPath*, including the project methodology, technology stack, system architecture, implementation, deployment, and results. It also explains how AWS services were configured and integrated to host the application successfully. The project serves as a practical example of cloud-native application deployment and demonstrates the knowledge and skills gained during the **Summer Online Internship – AWS Cloud Computing**.

2. METHODOLOGY

The development of **PlacementPath** followed a structured and systematic methodology to ensure the successful completion of the project objectives. The development process was divided into several phases, including requirements gathering, system design, implementation, database management, testing, deployment, and refinement. Each phase contributed to building a secure, scalable, and cloud-based full stack web application. The methodology also ensured that the application could be efficiently deployed on Amazon Web Services (AWS) using industry-standard practices.

2.1 Requirements Gathering

The first phase of the project involved gathering and analyzing the project requirements. The primary objective was to develop a centralized platform that helps students access placement-related resources from a single location. Existing placement preparation methods were studied to understand the challenges faced by students, such as scattered learning resources, difficulty accessing company information, and the absence of a unified platform.

Based on these observations, both functional and non-functional requirements were identified. The application was required to provide secure user registration and login, display placement resources and company information, and maintain user data securely. In addition, the system was expected to be responsive, reliable, secure, and easily accessible through cloud deployment.

2.2 System Design

After gathering the requirements, the overall system architecture was designed. The application follows a ***three-tier architecture***, consisting of the presentation layer, application layer, and data layer. This architecture separates the user interface, business logic, and database, making the application easier to maintain and scale.

The frontend was designed using HTML, CSS, and JavaScript to provide an interactive and responsive user experience. The backend was developed using the Flask framework in Python, which processes user requests, performs authentication, and communicates with the database. Amazon RDS using the MySQL engine was selected as the production database for secure and reliable data storage.

2.3 Implementation

During the implementation phase, both the frontend and backend components were developed and integrated. The frontend pages were created to provide a clean and user-friendly interface for students. Features such as user registration, login, placement resources, company information, and preparation roadmaps were implemented.

The backend was developed using Python and Flask to handle API requests, validate user inputs, manage authentication using JSON Web Tokens (JWT), and perform database operations. The application logic was tested continuously during

development to ensure proper communication between the frontend and backend components.

2.4 Database Management

The application uses **Amazon RDS** with the **MySQL** database engine to store application data securely in the cloud. During local development and testing, SQLite was used for simplicity, while the final deployed application was connected to Amazon RDS for production use.

The database stores user registration details and other application-related information. Flask communicates directly with the MySQL database using database connectors, allowing secure storage and retrieval of data. Proper validation was performed before inserting or updating records to maintain data integrity and consistency.

2.5 Testing

The application was tested throughout the development process to verify its functionality, reliability, and security. Individual components such as user registration, login, database connectivity, and frontend responsiveness were tested separately before performing complete system testing.

Authentication mechanisms, including JWT token generation and validation, were also tested to ensure that only authenticated users could access protected resources. Database operations were verified by connecting directly to the Amazon RDS MySQL database and confirming that user information was stored correctly. Additional testing was performed to ensure compatibility across different web browsers and screen sizes.

2.6 Deployment

After successful testing, the application was deployed on **Amazon Web Services (AWS)** using multiple cloud services. The static frontend was hosted on **Amazon S3**, while the backend Flask application was deployed on an **Amazon EC2** instance running Ubuntu Linux. **Nginx** was configured as the reverse proxy, and **Gunicorn** was used as the WSGI application server to run the Flask application.

The production database was hosted on **Amazon RDS (MySQL)**. **Amazon CloudFront** was configured with two origins: one origin connected to Amazon S3 for delivering static website files and another origin connected to the EC2 instance for handling

backend API requests. **AWS IAM** was used to securely manage permissions and access to AWS resources. This deployment architecture provides improved availability, security, and performance.

2.7 Iterative Refinement

Throughout the project development, continuous improvements were made based on testing results and implementation experience. User interface elements were refined to improve usability and navigation. Backend logic was optimized to improve request handling and database communication. Security measures, including password hashing and JWT authentication, were strengthened to enhance application security.

Deployment configurations were also refined by optimizing Nginx, Gunicorn, CloudFront behaviors, and database connectivity to ensure stable application performance. These iterative improvements resulted in a reliable, secure, and efficient cloud-based application that meets the objectives of the project

Conclusion of Methodology

The methodology adopted for the development of PlacementPath ensured a systematic approach to designing, implementing, testing, and deploying the application. By following structured development practices and integrating multiple AWS services, the project successfully demonstrates the practical implementation of cloud computing concepts through a real-world full stack web application. The methodology also provided valuable hands-on experience in modern web development, cloud deployment, and secure application management.

3. TECHNOLOGY STACK

PlacementPath was developed using a modern full stack technology stack integrated with Amazon Web Services (AWS). The selected technologies provide a secure, scalable, and reliable platform for developing and deploying cloud-based web applications. The frontend, backend, database, web server, and cloud infrastructure work together to deliver a responsive and efficient user experience. The following technologies and services were used during the development and deployment of the project.

3.1 Cloud Platform

Amazon Web Services (AWS):

Amazon Web Services (AWS) is the cloud platform used to host and deploy the PlacementPath application. AWS provides reliable, scalable, and secure cloud services that enable the application to be accessed over the internet. The project utilizes multiple AWS services for hosting, storage, database management, content delivery, and access management.

3.2 AWS Services

Amazon EC2 (Elastic Compute Cloud):

Amazon EC2 provides the virtual server on which the Flask backend application is deployed. The EC2 instance runs Ubuntu Linux and hosts the backend using Nginx and Unicorn. It processes user requests, executes business logic, and communicates with the database.

Amazon S3 (Simple Storage Service):

Amazon S3 is used to host the static frontend of the application. HTML, CSS, JavaScript, images, and other static assets are stored in an S3 bucket configured for static website hosting, providing reliable and highly available storage.

Amazon RDS (Relational Database Service):

Amazon RDS with the MySQL database engine is used as the production database. It securely stores user registration details and application data while providing managed database services such as automated maintenance, backup support, and high availability.

Amazon CloudFront:

Amazon CloudFront is used as the Content Delivery Network (CDN) for the application. It is configured with two origins—Amazon S3 for static website content and Amazon EC2 for backend API requests. This enables efficient delivery of both static and dynamic content while improving application performance and reducing latency.

AWS Identity and Access Management (IAM):

AWS IAM is used to securely manage users, permissions, and access policies for AWS resources. It ensures that only authorized users and services can access the required AWS resources while following security best practices.

3.3 Frontend Technologies

HTML5:

HTML5 is used to create the structure and content of the web pages. It defines the layout of the application and provides semantic elements for better organization and accessibility.

CSS3:

CSS3 is used to design the user interface by applying styles, colors, layouts, spacing, and responsive design. It improves the visual appearance of the application across different devices.

JavaScript:

JavaScript adds interactivity and dynamic behavior to the frontend. It is used for form validation, user interactions, API communication, and updating web pages without requiring full page reloads.

3.4 Backend Technologies

Python:

Python is the primary programming language used to develop the backend application. Its simplicity, readability, and extensive library support make it suitable for web application development.

Flask Framework:

Flask is a lightweight Python web framework used to build the backend APIs and application logic. It handles routing, request processing, user authentication, database communication, and response generation.

3.5 Database

MySQL (Amazon RDS):

MySQL is the relational database management system used in the project through Amazon RDS. It stores user information and application data in structured tables while supporting efficient querying, data integrity, and secure storage.

3.6 Web Server and Application Server

Nginx:

Nginx is configured as the reverse proxy web server. It receives incoming HTTP requests, serves static content when required, and forwards backend requests to the Flask application running through Gunicorn.

Gunicorn:

Gunicorn (Green Unicorn) is the WSGI application server used to run the Flask application on the EC2 instance. It efficiently handles multiple client requests and works together with Nginx to provide stable application performance.

3.7 Authentication and Security

JSON Web Token (JWT):

JWT is used to implement secure user authentication. After successful login, a token is generated and used to verify the identity of users when accessing protected resources.

Password Hashing:

User passwords are securely stored using password hashing techniques before being saved in the database. This enhances application security by preventing the storage of plain-text passwords.

3.8 Development Tools

The application was developed using **Visual Studio Code** as the primary code editor. **Git** was used for version control to manage source code changes during development, while **GitHub** was used to store and maintain the project repository. During local development, **SQLite** was used for initial testing before migrating the application to **Amazon RDS (MySQL)** for production deployment.

4. SYSTEM DESIGN / ARCHITECTURE

The PlacementPath application is designed as a cloud-based full stack web application deployed on Amazon Web Services (AWS). The system follows a **three-tier architecture**, consisting of the Presentation Tier, Application Tier, and Data Tier. The frontend is hosted on Amazon S3, the backend is deployed on an Amazon EC2 instance, and the

application data is stored in Amazon RDS using the MySQL database engine. Amazon CloudFront is configured with two origins to efficiently deliver both static website content and backend API requests, while AWS IAM manages secure access to cloud resources.

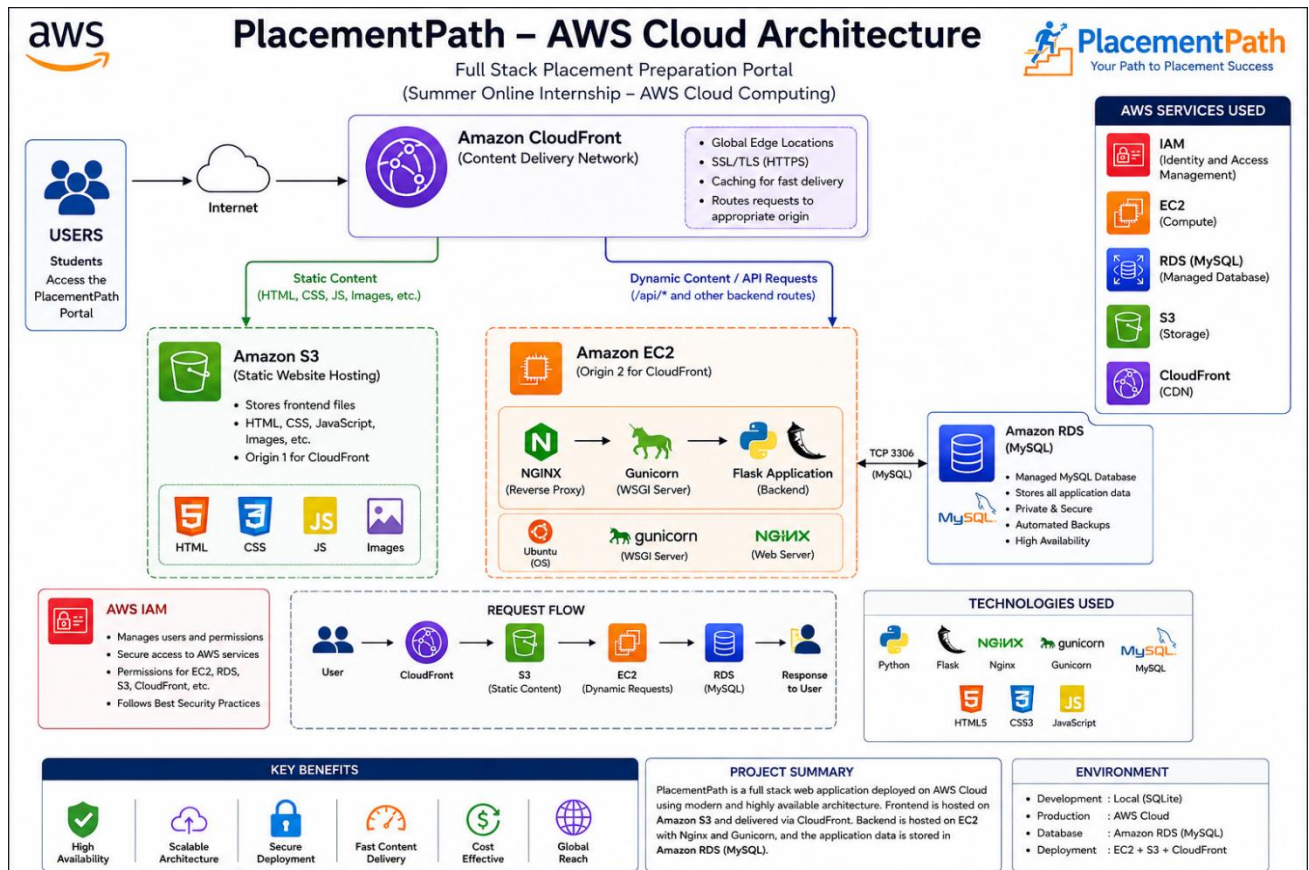


Figure 1. Overall AWS Architecture of PlacementPath

4.1 Presentation Tier – Amazon S3

The Presentation Tier is responsible for providing the user interface of the application. The frontend was developed using HTML, CSS, and JavaScript and is hosted as a static website on Amazon S3. Hosting the frontend on S3 provides high availability, durability, and easy access over the internet. Static assets such as HTML pages, CSS files, JavaScript files, and images are stored in the S3 bucket and served to users through Amazon CloudFront.

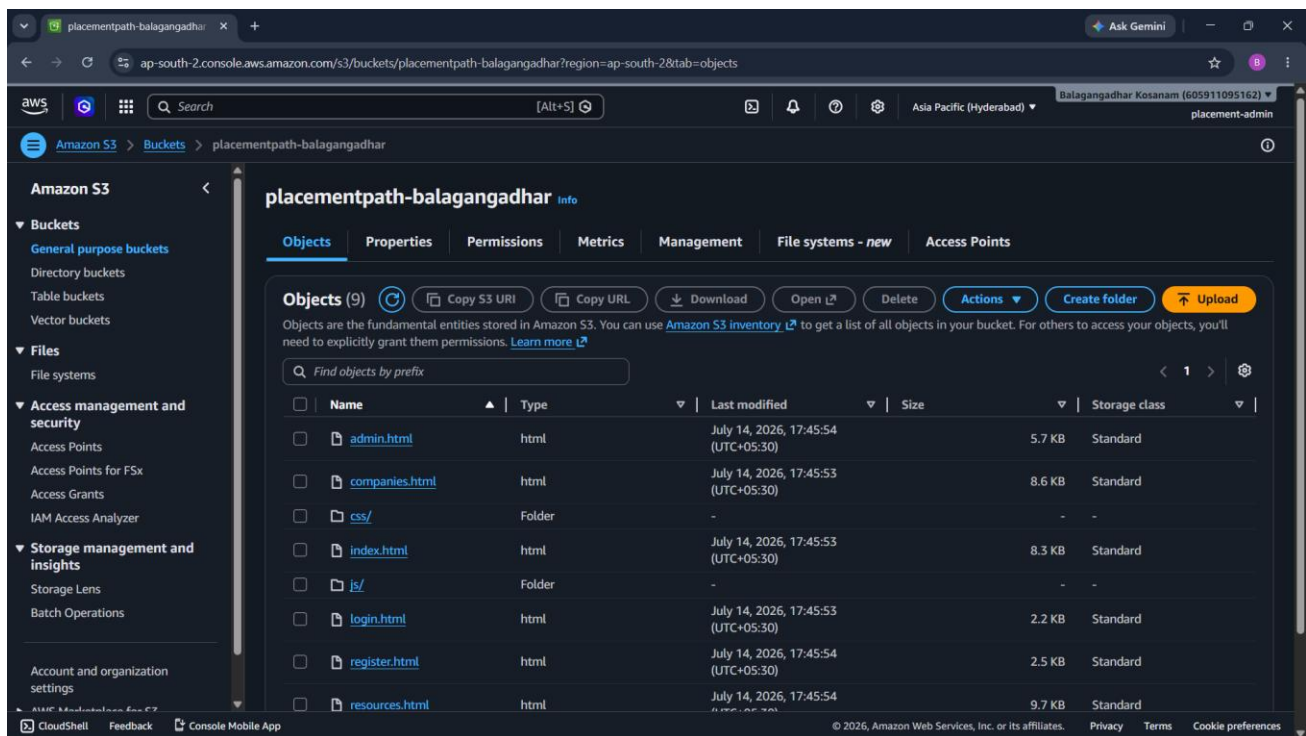


Figure 2: Amazon S3 Bucket showing the PlacementPath bucket.

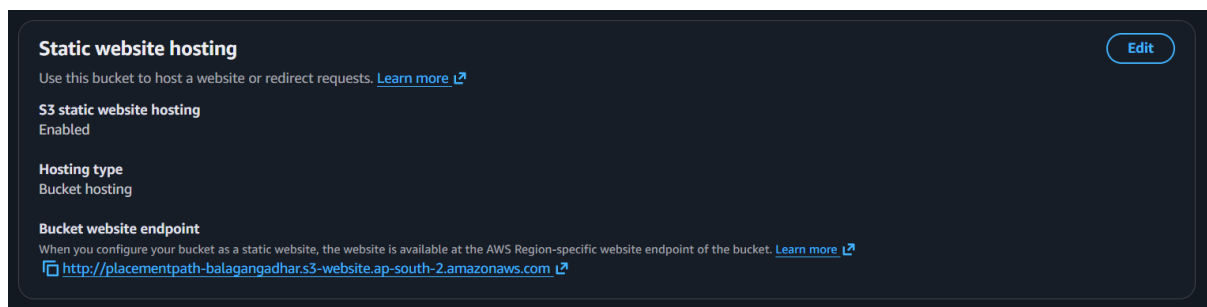


Figure 3: Static Website Hosting configuration of the S3 bucket.

4.2 Application Tier – Amazon EC2

The Application Tier contains the business logic of the application. The Flask backend is deployed on an Amazon EC2 instance running Ubuntu Linux. The EC2 instance receives API requests from CloudFront, processes user requests, performs authentication, communicates with the database, and returns responses to the frontend.

To improve performance and reliability, Nginx is configured as a reverse proxy, while Gunicorn acts as the WSGI application server that runs the Flask application.

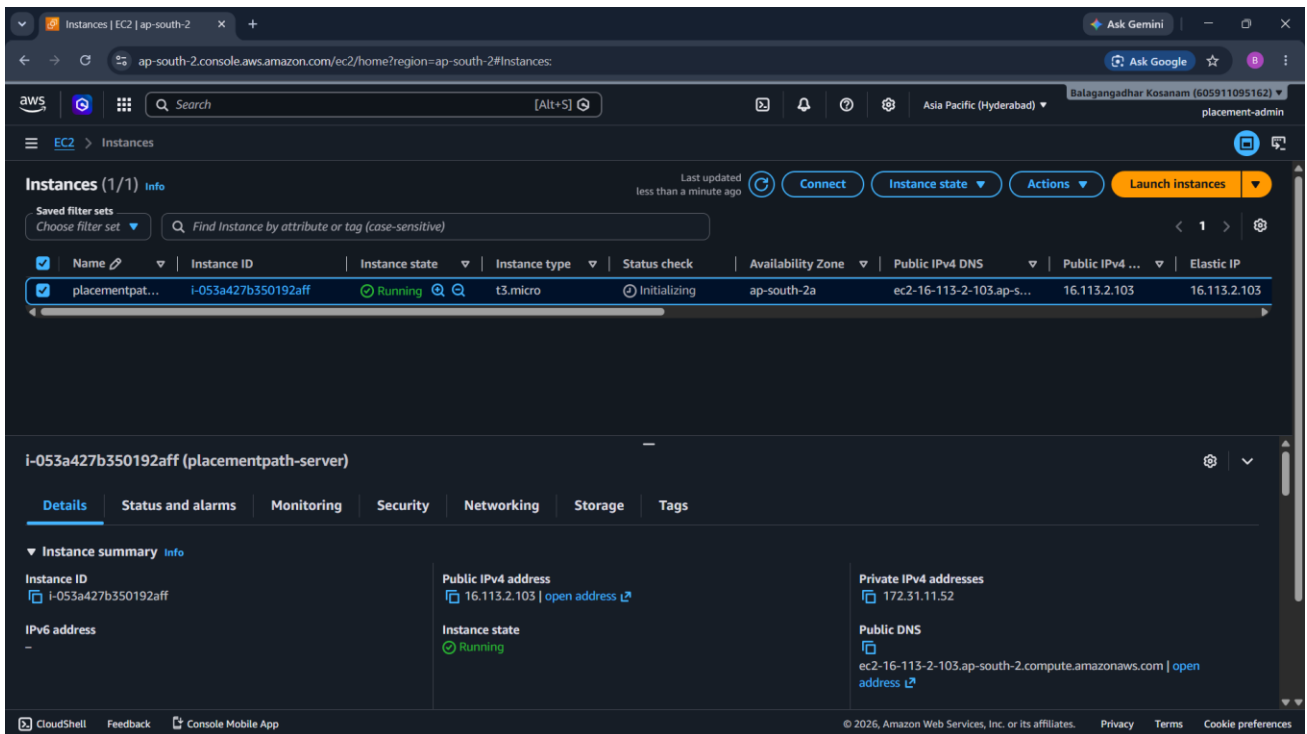


Figure 4: EC2 Instances page showing the running instance.

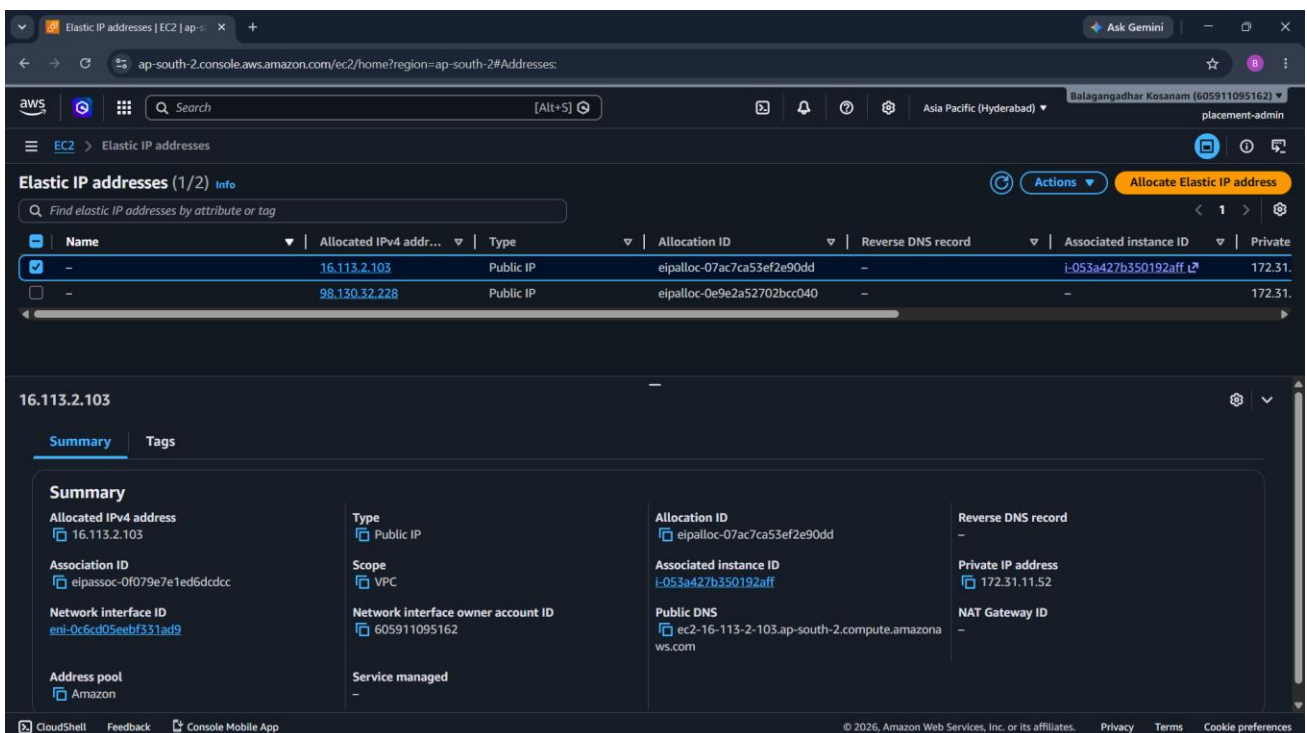


Figure 5: EC2 Instance Details showing the Elastic IP and instance status.

```

ubuntu@ip-172-31-11-52:~/PlacementPath/backend$ sudo systemctl status placementpath
● placementpath.service - PlacementPath Flask Backend
   Loaded: loaded (/etc/systemd/system/placementpath.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-07-15 08:31:38 UTC; 11min ago
     Main PID: 526 (gunicorn)
        Tasks: 4 (limit: 1013)
      Memory: 79.9M (peak: 80.2M)
         CPU: 1.008s
       CGroup: /system.slice/placementpath.service
               └─526 /home/ubuntu/PlacementPath/backend/venv/bin/python3 /home/ubuntu/PlacementPath/backend/venv/bin/gunicorn --workers 3 --bind 127.0.0.1:5000 app:app
                 └─775 /home/ubuntu/PlacementPath/backend/venv/bin/python3 /home/ubuntu/PlacementPath/backend/venv/bin/gunicorn --workers 3 --bind 127.0.0.1:5000 app:app
                   └─788 /home/ubuntu/PlacementPath/backend/venv/bin/python3 /home/ubuntu/PlacementPath/backend/venv/bin/gunicorn --workers 3 --bind 127.0.0.1:5000 app:app
                     └─801 /home/ubuntu/PlacementPath/backend/venv/bin/python3 /home/ubuntu/PlacementPath/backend/venv/bin/gunicorn --workers 3 --bind 127.0.0.1:5000 app:app

Jul 15 08:31:38 ip-172-31-11-52 systemd[1]: Started placementpath.service - PlacementPath Flask Backend.
Jul 15 08:31:39 ip-172-31-11-52 gunicorn[526]: [2026-07-15 08:31:39 +0000] [526] [INFO] Starting gunicorn 23.0.0
Jul 15 08:31:39 ip-172-31-11-52 gunicorn[526]: [2026-07-15 08:31:39 +0000] [526] [INFO] Listening at: http://127.0.0.1:5000 (526)
Jul 15 08:31:39 ip-172-31-11-52 gunicorn[526]: [2026-07-15 08:31:39 +0000] [526] [INFO] Using worker: sync
Jul 15 08:31:39 ip-172-31-11-52 gunicorn[775]: [2026-07-15 08:31:39 +0000] [775] [INFO] Booting worker with pid: 775
Jul 15 08:31:39 ip-172-31-11-52 gunicorn[788]: [2026-07-15 08:31:39 +0000] [788] [INFO] Booting worker with pid: 788
Jul 15 08:31:39 ip-172-31-11-52 gunicorn[801]: [2026-07-15 08:31:39 +0000] [801] [INFO] Booting worker with pid: 801

```

Figure 6: Terminal screenshot showing Gunicorn service running.

```

ubuntu@ip-172-31-11-52:~/PlacementPath/backend$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-07-15 08:31:38 UTC; 12min ago
     Docs: man:nginx(8)
     Main PID: 606 (nginx)
        Tasks: 3 (limit: 1013)
      Memory: 3.9M (peak: 4.3M)
         CPU: 28ms
       CGroup: /system.slice/nginx.service
               └─606 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
                 └─607 "nginx: worker process"
                   └─608 "nginx: worker process"

Jul 15 08:31:38 ip-172-31-11-52 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Jul 15 08:31:38 ip-172-31-11-52 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.

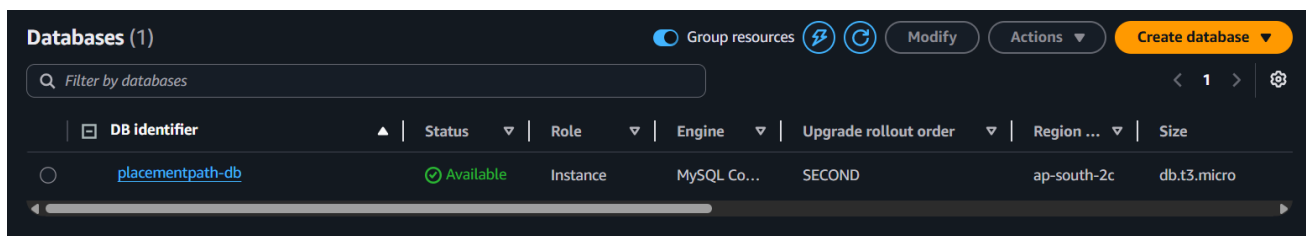
```

Figure 7: Terminal screenshot showing Nginx service status.

4.3 Data Tier – Amazon RDS (MySQL)

The Data Tier stores all application data securely. Amazon RDS with the MySQL engine is used as the production database. It stores user registration details and other application-related data while providing a managed and secure database environment.

The Flask application connects directly to Amazon RDS to perform database operations such as inserting user information and retrieving data during login and other application requests.



DB identifier	Status	Role	Engine	Upgrade rollout order	Region	Size
placementpath-db	Available	Instance	MySQL Co...	SECOND	ap-south-2c	db.t3.micro

Figure 8: Amazon RDS Dashboard showing the MySQL database instance.

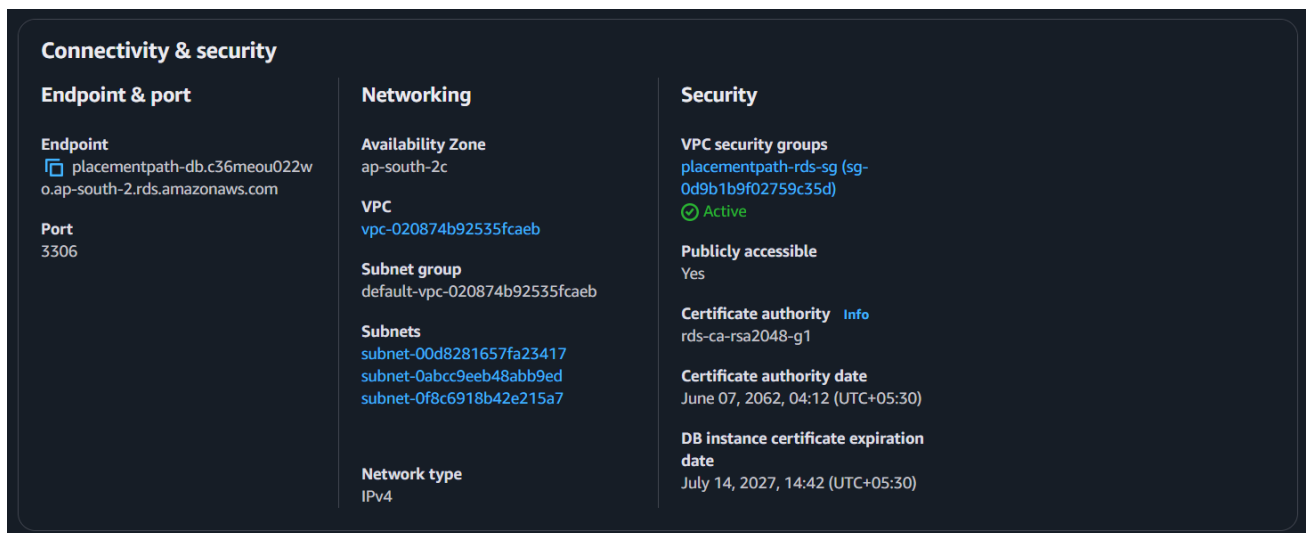


Figure 9: RDS Configuration page showing endpoint and engine details.

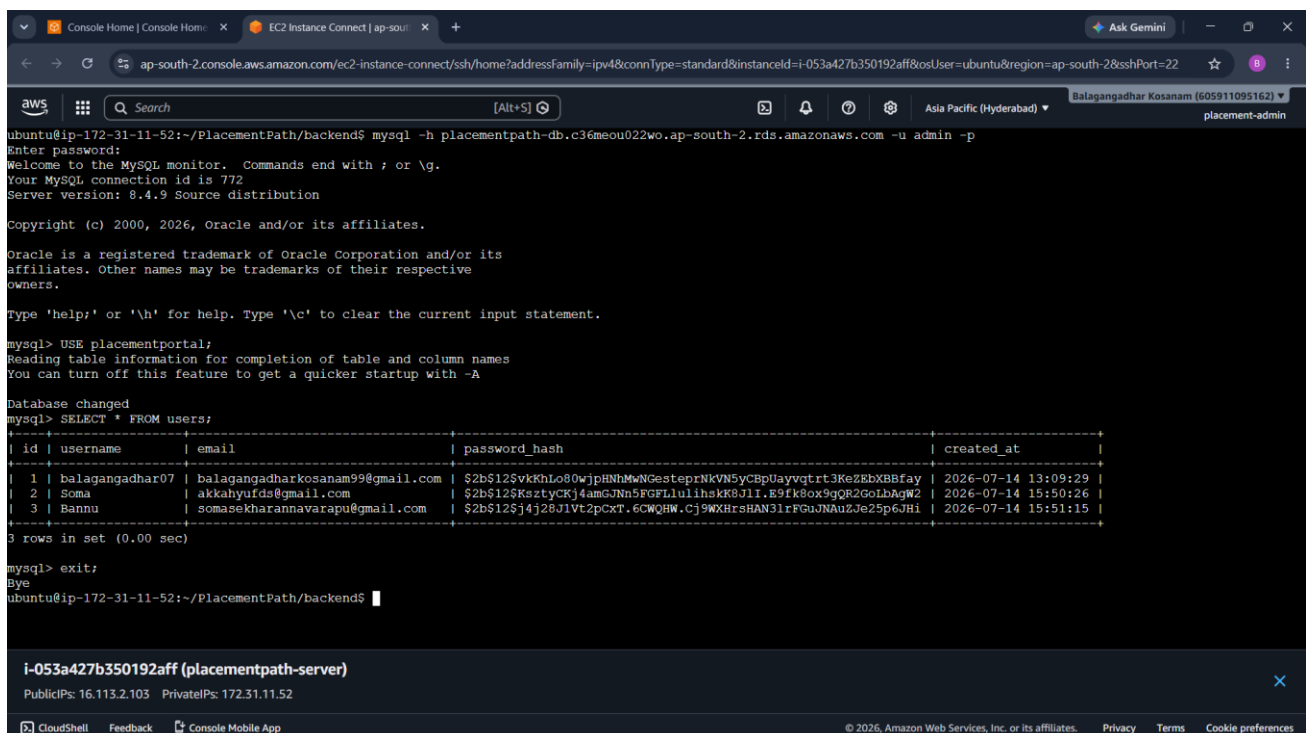


Figure 10. Accessing Amazon RDS MySQL Database through EC2 and Displaying Registered User Records

4.4 Content Delivery Network – Amazon CloudFront

Amazon CloudFront is used as the Content Delivery Network (CDN) to improve application performance and reduce response time. The CloudFront distribution is configured with **two origins**.

The first origin is connected to the Amazon S3 bucket, which serves the frontend static website files. The second origin is connected to the Amazon EC2 instance, which handles backend API requests. This configuration allows both static and dynamic content to be delivered through a single CloudFront distribution.

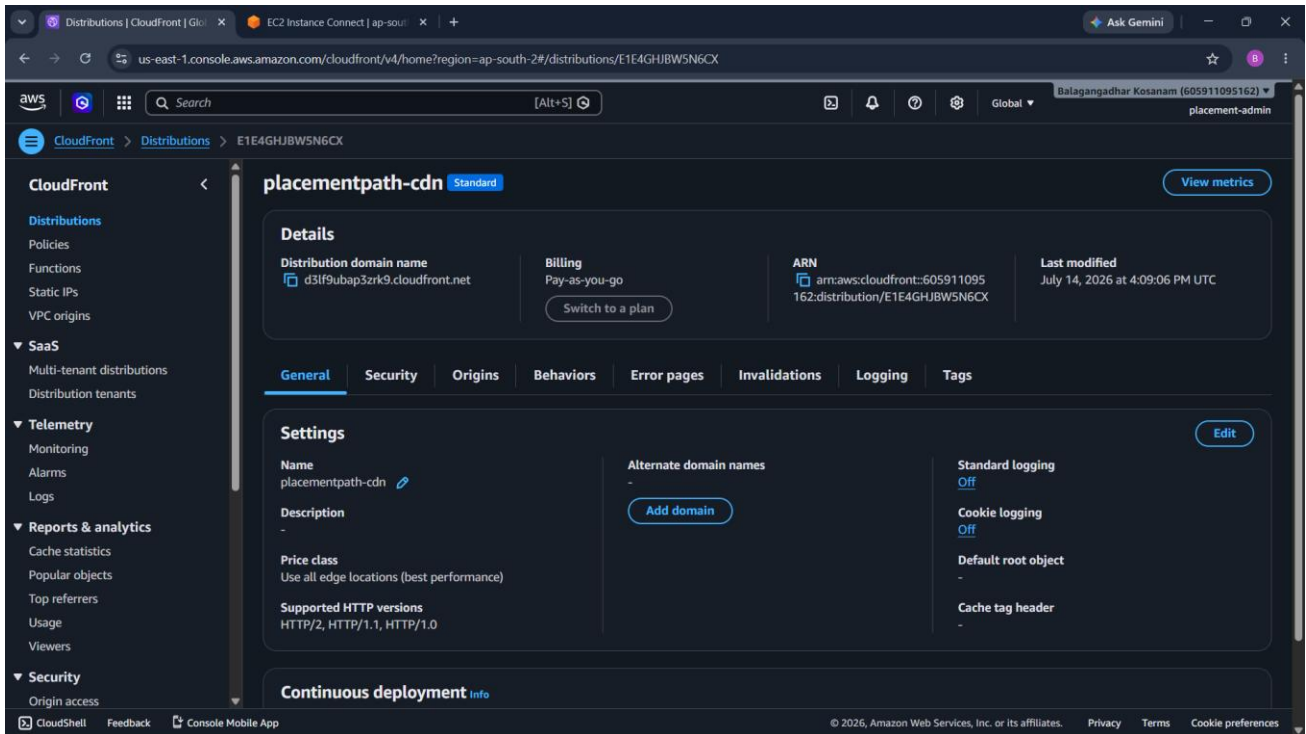


Figure 11: CloudFront Distribution page.

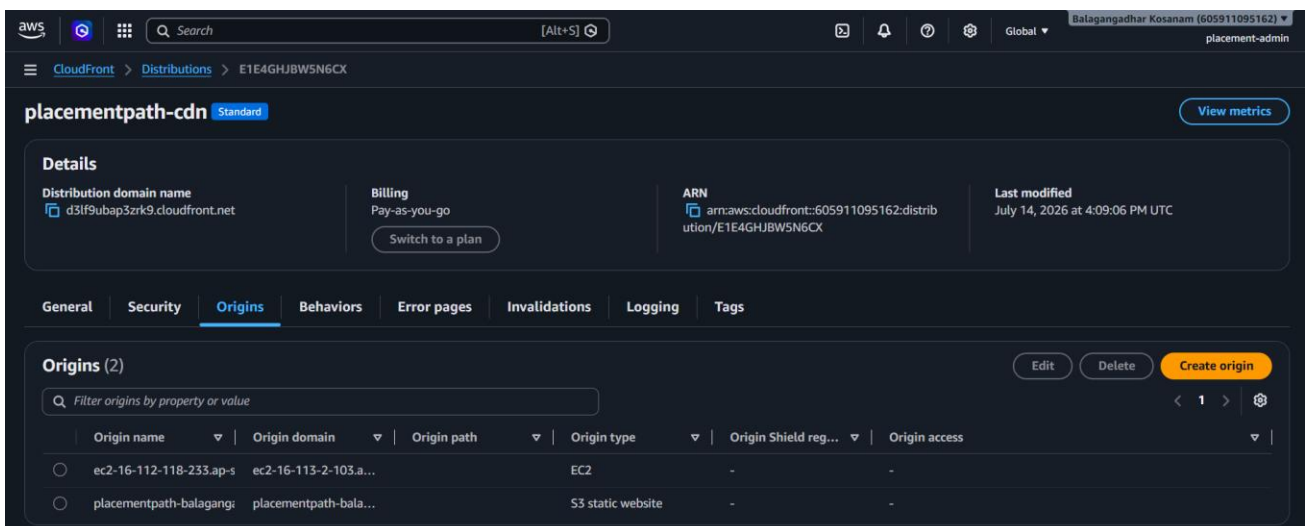


Figure 12: CloudFront Origins page showing S3 Origin and EC2 Origin

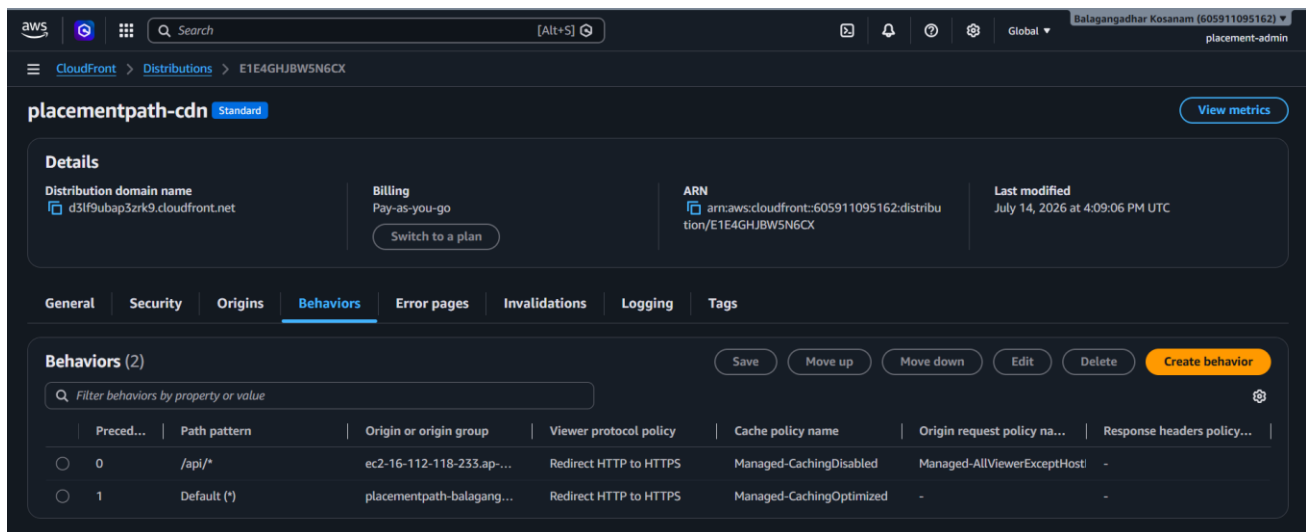


Figure 13: CloudFront Behaviors page showing request routing/path patterns, if configured.

4.5 Identity and Access Management (IAM)

AWS Identity and Access Management (IAM) is used to securely manage access to AWS resources. IAM users and permissions ensure that only authorized users can manage services such as EC2, S3, RDS, and CloudFront. This enhances the overall security of the cloud infrastructure.

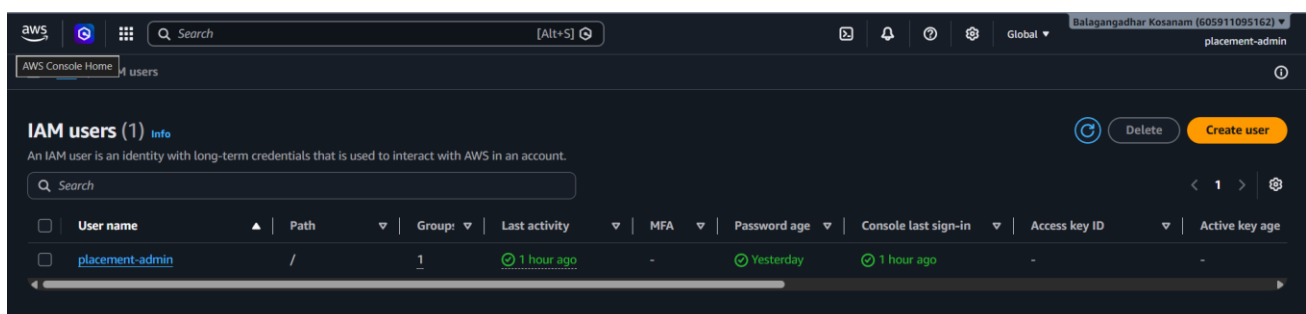


Figure 14: IAM Users page.

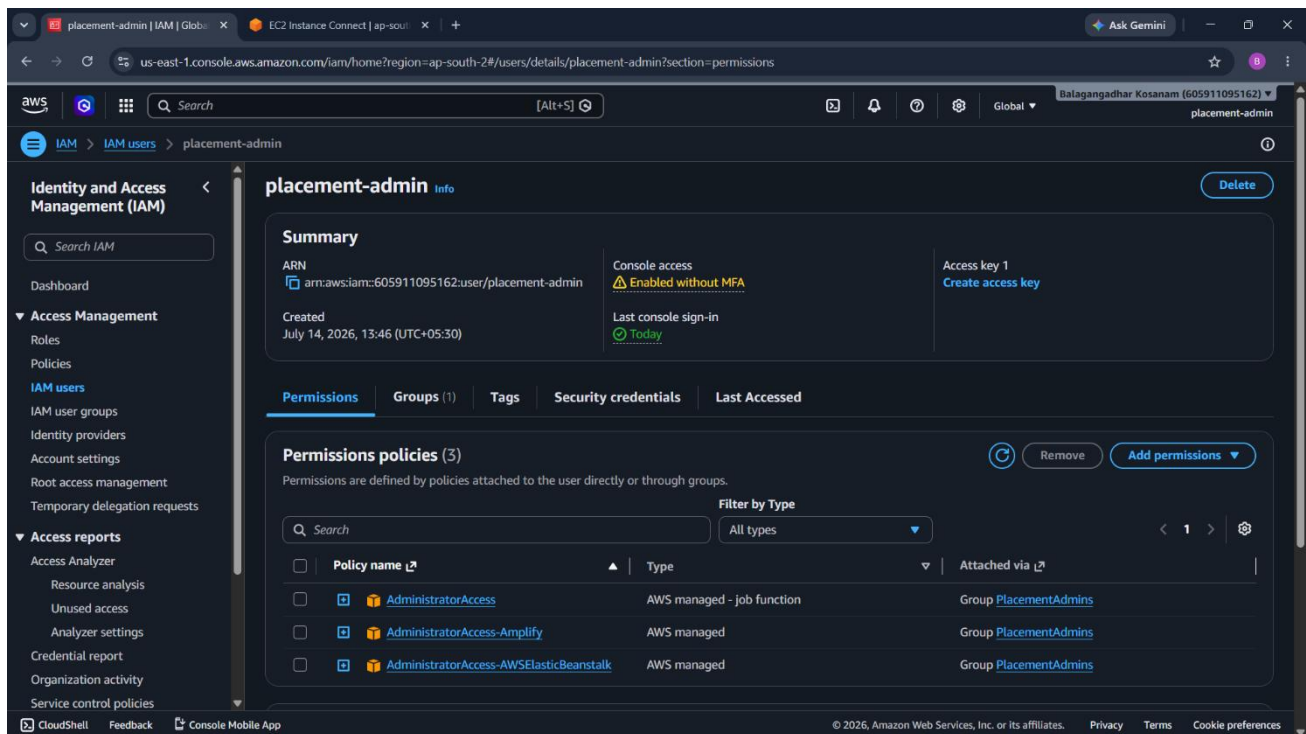


Figure 15: IAM Policies or User Permissions page.

4.6 System Workflow

The PlacementPath application processes requests through a sequence of AWS services. When a user accesses the application, CloudFront determines whether the request is for static content or backend processing. Static files are served from Amazon S3, while API requests are forwarded to the EC2 instance. The Flask application processes the request and interacts with Amazon RDS when database operations are required. The response is then returned to the user through CloudFront.

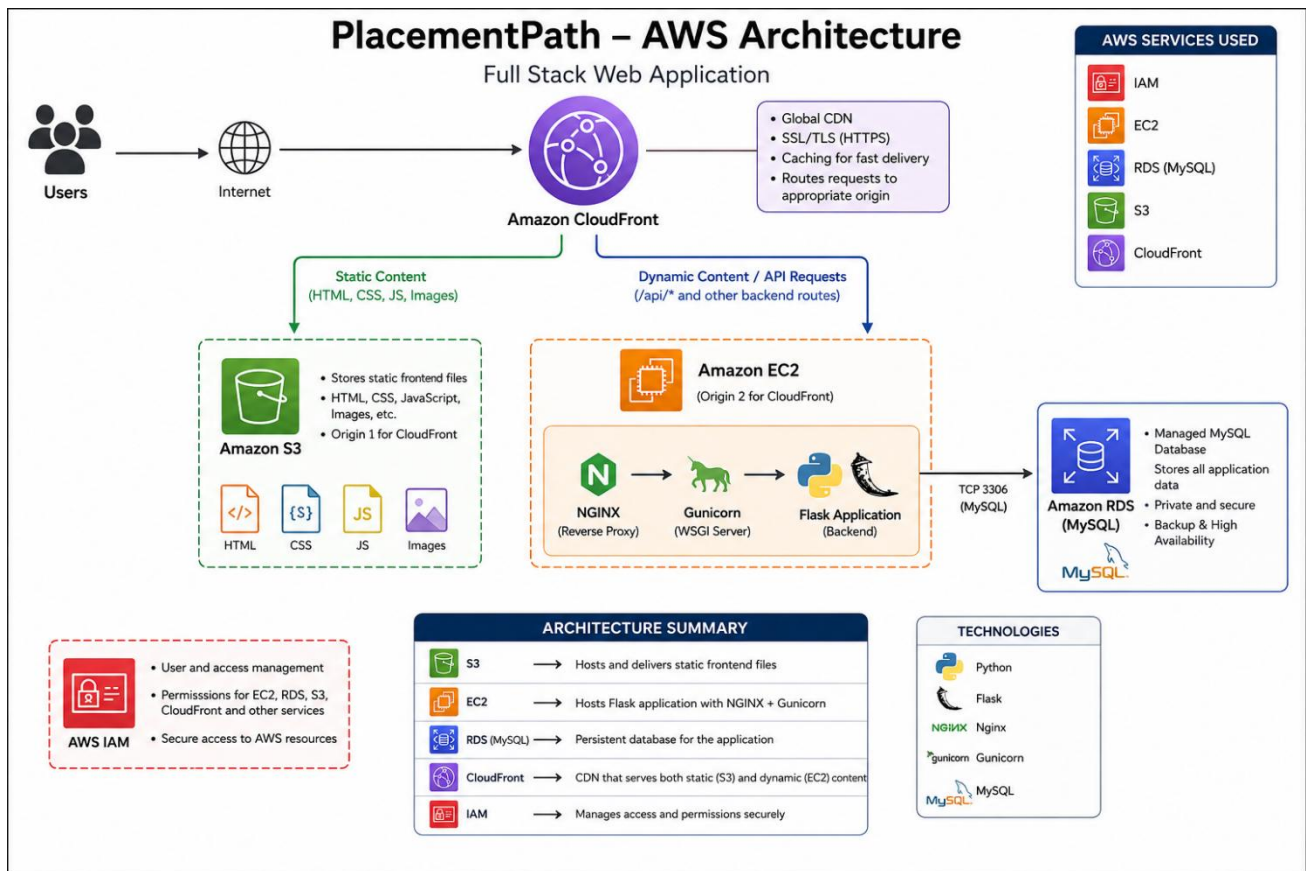


Figure 16: Final application architecture diagram.

4.7 Security Features

Security is an important aspect of the PlacementPath application. User authentication is implemented using JSON Web Tokens (JWT), and passwords are stored using secure hashing techniques. AWS IAM controls access to cloud resources, while EC2 Security Groups are configured to allow only the required network traffic. These security measures help protect both application data and cloud infrastructure.

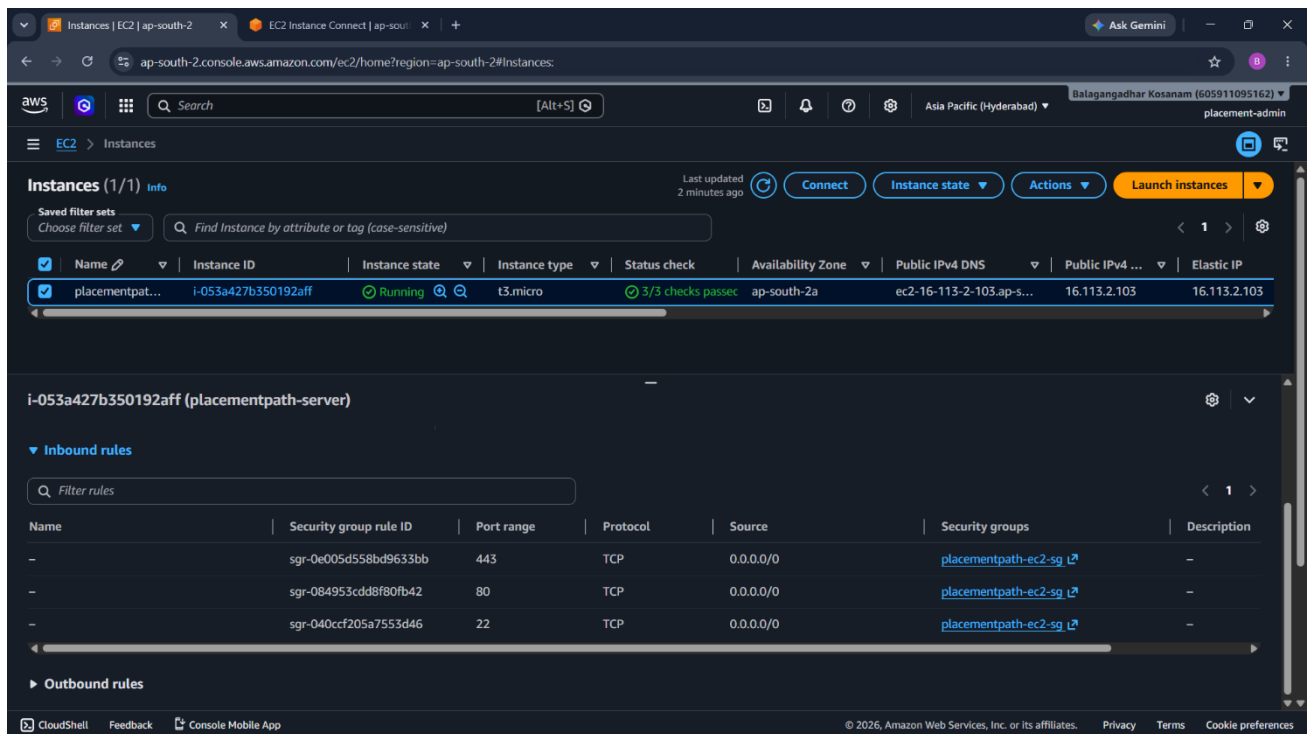


Figure 17: EC2 Security Group inbound rules.

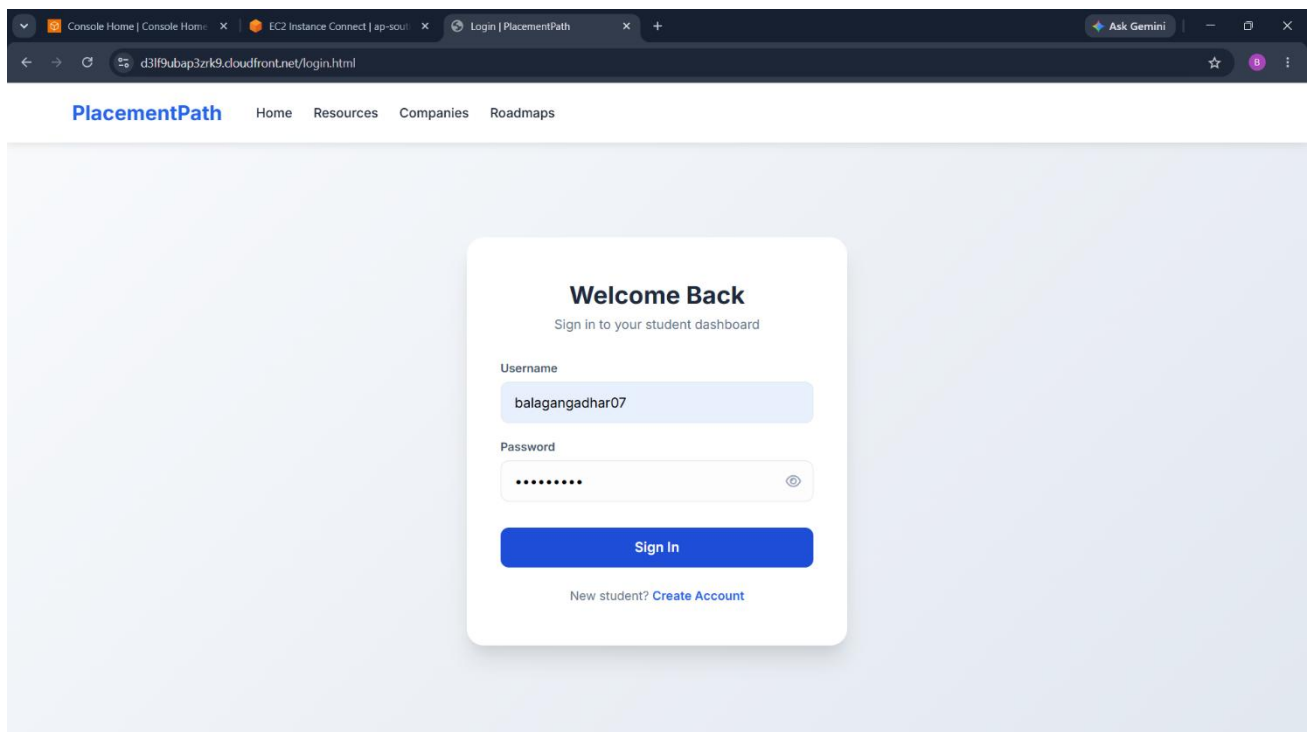


Figure 18: Secure User Login Page

5. IMPLEMENTATION

The implementation of ***PlacementPath*** was carried out in a systematic manner by integrating frontend development, backend development, database management, and AWS cloud services. The following steps describe the implementation process.

5.1 Frontend Development

- Developed the user interface using ***HTML5, CSS3, and JavaScript***.
- Designed responsive web pages for better user experience.
- Created Home, Register, Login, Resources, Companies, and Roadmaps pages.
- Implemented client-side form validation using JavaScript.
- Integrated frontend pages with backend APIs.

5.2 Backend Development

- Developed the backend using ***Python*** and the ***Flask*** framework.
- Implemented routing and request handling.
- Developed REST API endpoints for user operations.
- Implemented user registration and login functionality.
- Added JWT-based authentication.
- Secured user passwords using password hashing.

5.3 Database Integration

- Created the production database using ***Amazon RDS (MySQL)***.
- Connected the Flask application to the RDS database.
- Stored user registration details in the database.
- Retrieved user information during login verification.
- Performed database operations using SQL queries.

5.4 Backend Deployment on Amazon EC2

- Launched an Ubuntu EC2 instance.
- Connected to the EC2 instance using SSH.
- Uploaded the backend project files.
- Created a Python virtual environment.
- Installed all required Python dependencies.
- Configured the Flask application for production deployment.

5.5 Configuring Nginx and Gunicorn

- Installed Gunicorn as the WSGI application server.
- Installed Nginx as the reverse proxy server.
- Configured Gunicorn to run the Flask application.
- Configured Nginx to forward API requests to Gunicorn.
- Verified both services were running successfully.

5.6 Frontend Deployment on Amazon S3

- Created an Amazon S3 bucket.
- Enabled static website hosting.
- Uploaded HTML, CSS, JavaScript, and image files.
- Verified successful hosting of the frontend application.

5.7 Amazon CloudFront Configuration

- Created a CloudFront distribution.
- Configured **Amazon S3** as the first origin.
- Configured **Amazon EC2** as the second origin.

- Configured cache behaviors for routing requests.
- Enabled HTTPS content delivery.
- Verified successful communication between frontend and backend.

5.8 Security Configuration

- Configured AWS IAM for secure resource access.
- Configured Security Groups for EC2 and RDS.
- Implemented JWT authentication.
- Applied password hashing for user credentials.
- Allowed only authorized access to protected APIs.

5.9 System Testing

- Tested user registration functionality.
- Tested user login functionality.
- Verified database connectivity with Amazon RDS.
- Tested frontend and backend integration.
- Verified CloudFront content delivery.
- Confirmed successful deployment of the complete application.

Implementation Summary

The successful implementation of **PlacementPath** demonstrates the practical development and deployment of a cloud-based full stack web application using Amazon Web Services (AWS). The application was developed using HTML, CSS, JavaScript, Python, and the Flask framework, following a three-tier architecture that separates the presentation, application, and data layers. The frontend was hosted on Amazon S3, while the backend was deployed on an Amazon EC2 instance using Nginx and Gunicorn for efficient request handling. Application data was securely stored in

Amazon RDS using the MySQL database engine, and Amazon CloudFront was configured with separate origins for the frontend and backend to provide efficient content delivery. AWS IAM was used to manage secure access to cloud resources, while JWT authentication and password hashing enhanced application security. The successful integration of these technologies and AWS services resulted in a secure, scalable, and reliable web application, demonstrating the effective use of cloud computing concepts in a real-world deployment scenario.

6. RESULTS

The implementation of *PlacementPath* successfully achieved the objectives of developing and deploying a cloud-based full stack web application using Amazon Web Services (AWS). The application was tested under different scenarios to verify its functionality, security, cloud deployment, and overall performance. The following results demonstrate the successful integration of AWS services and the proper functioning of the application.

6.1 Successful User Registration

The user registration functionality was tested by creating new user accounts through the registration page. The application successfully validated user inputs and stored the registered user details in the Amazon RDS MySQL database. This confirmed proper communication between the frontend, backend, and database.

6.2 Secure User Authentication

The login functionality was successfully implemented using JSON Web Tokens (JWT). Registered users were able to log in securely using their credentials. The backend validated user information, generated authentication tokens, and allowed authenticated users to access the application.

6.3 Successful Cloud Deployment

The *PlacementPath* application was successfully deployed on Amazon Web Services using a three-tier architecture. The frontend was hosted on Amazon S3, the backend was deployed on an Amazon EC2 instance, and the database was hosted on Amazon

RDS. Amazon CloudFront was configured to deliver both static and dynamic content efficiently through separate origins.

6.4 Database Connectivity

The Flask application successfully established a connection with the Amazon RDS MySQL database. User registration data was stored correctly, and database operations such as data retrieval and authentication were performed without errors. The database was verified by accessing it securely through the EC2 instance.

6.5 CloudFront Content Delivery

Amazon CloudFront successfully delivered the frontend content from the Amazon S3 bucket and forwarded backend API requests to the Amazon EC2 instance. The two-origin configuration worked correctly, providing seamless communication between the frontend and backend while improving application accessibility.

6.6 Application Performance

The application responded successfully to user requests, including registration, login, and page navigation. The integration of Nginx, Gunicorn, and Flask on the EC2 instance ensured reliable request processing, while Amazon RDS provided stable database performance.

6.6 Screenshots of the Output.

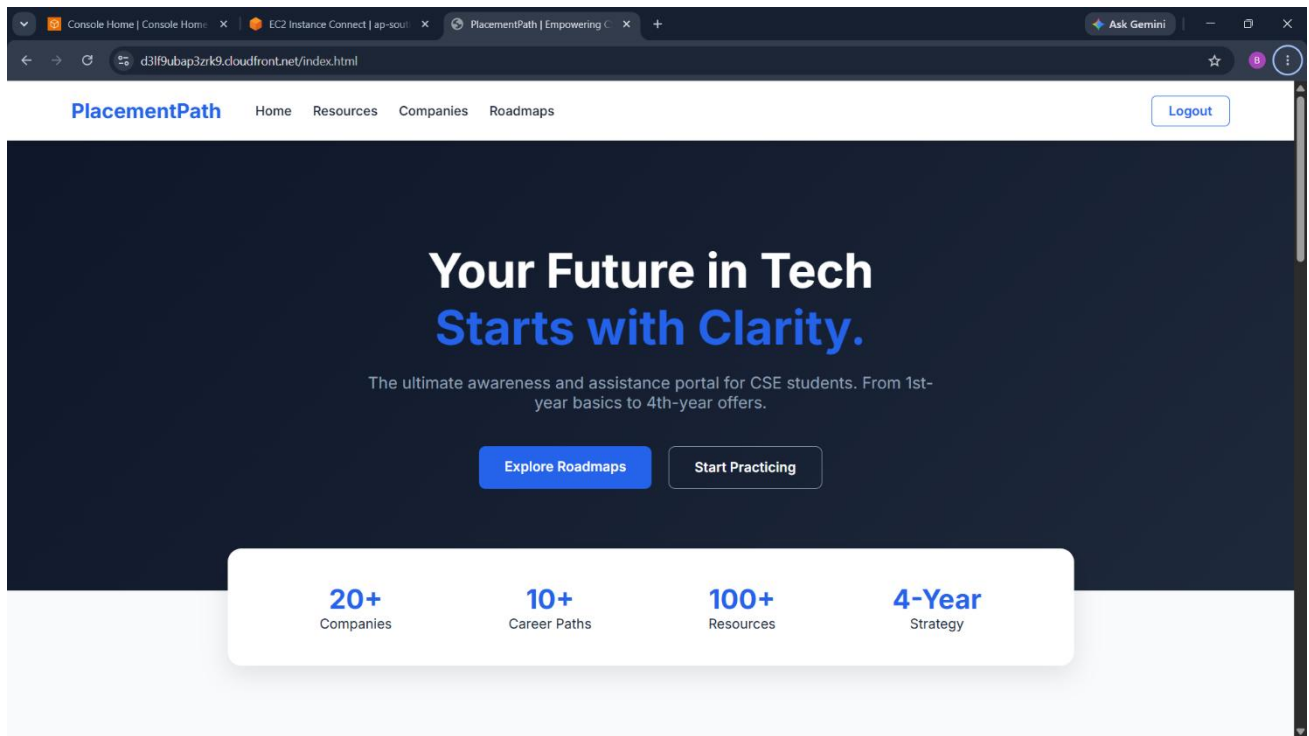


Figure 19. PlacementPath Home Page

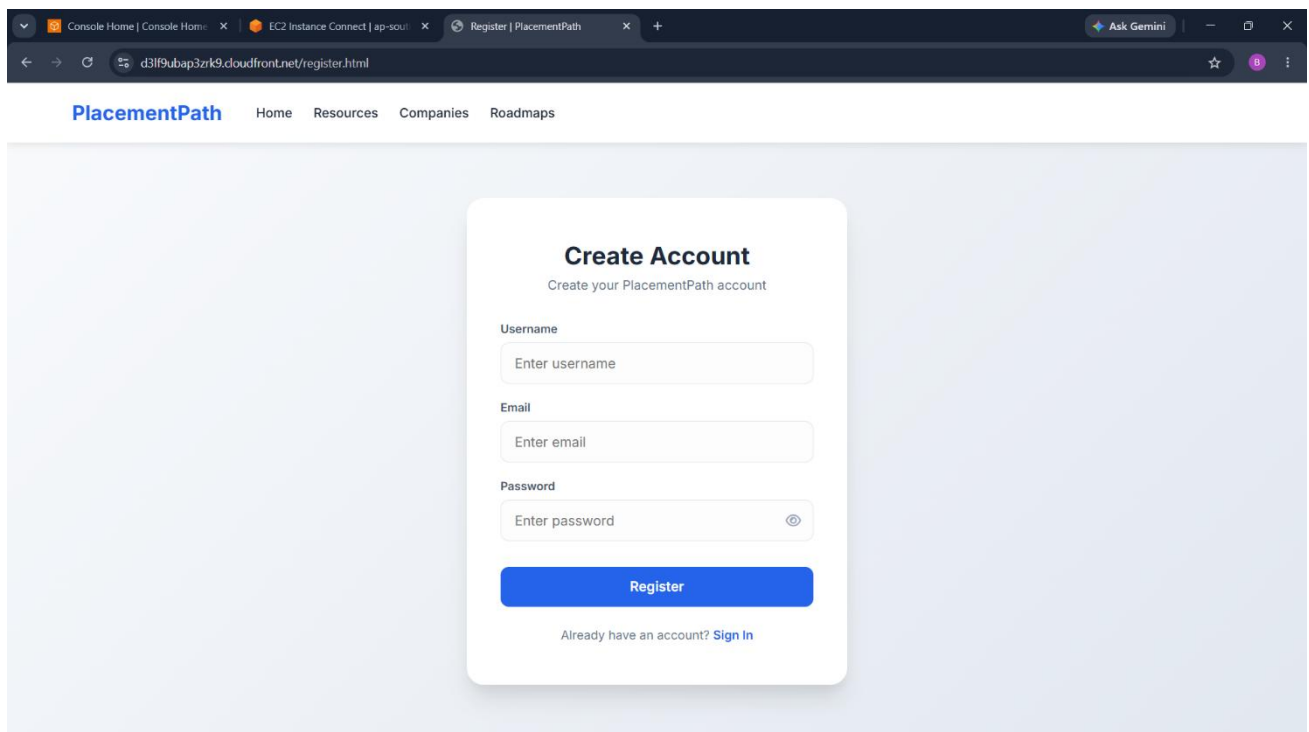


Figure 20. User Registration Page

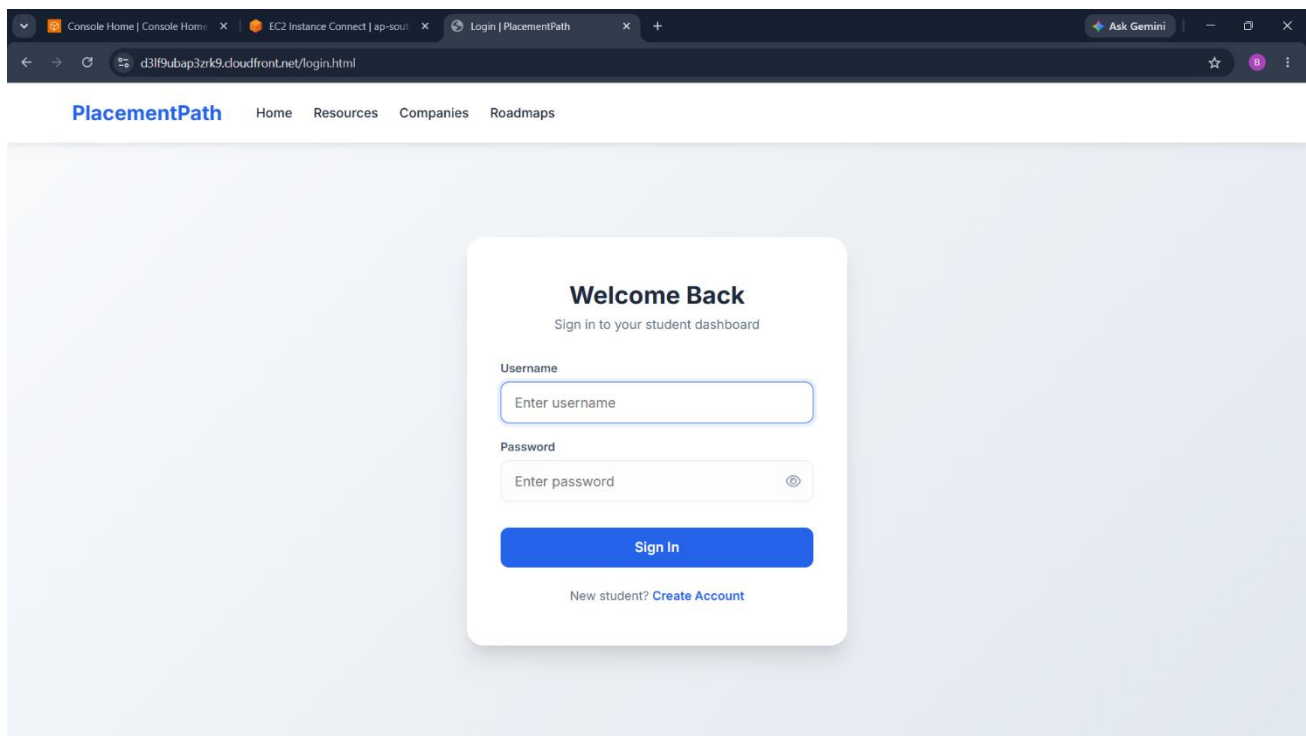


Figure 21. User Login Page

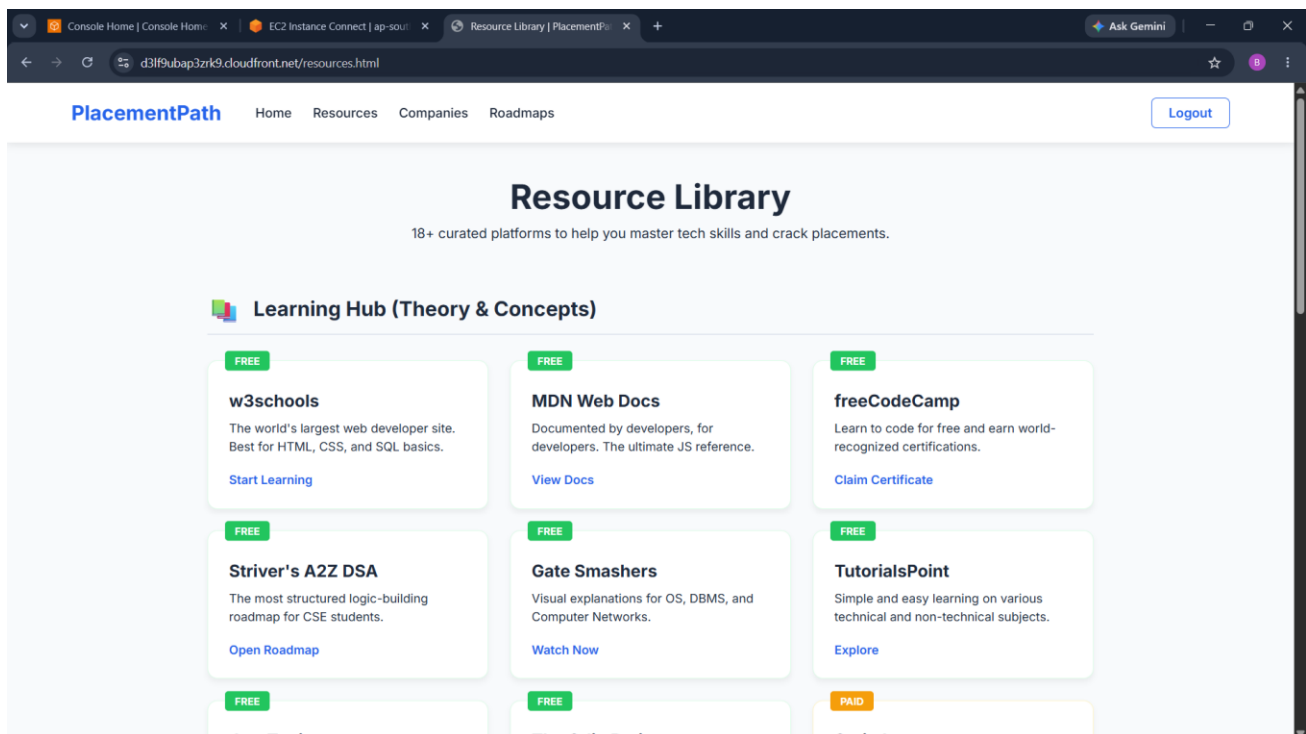


Figure 22. Placement Resources Page

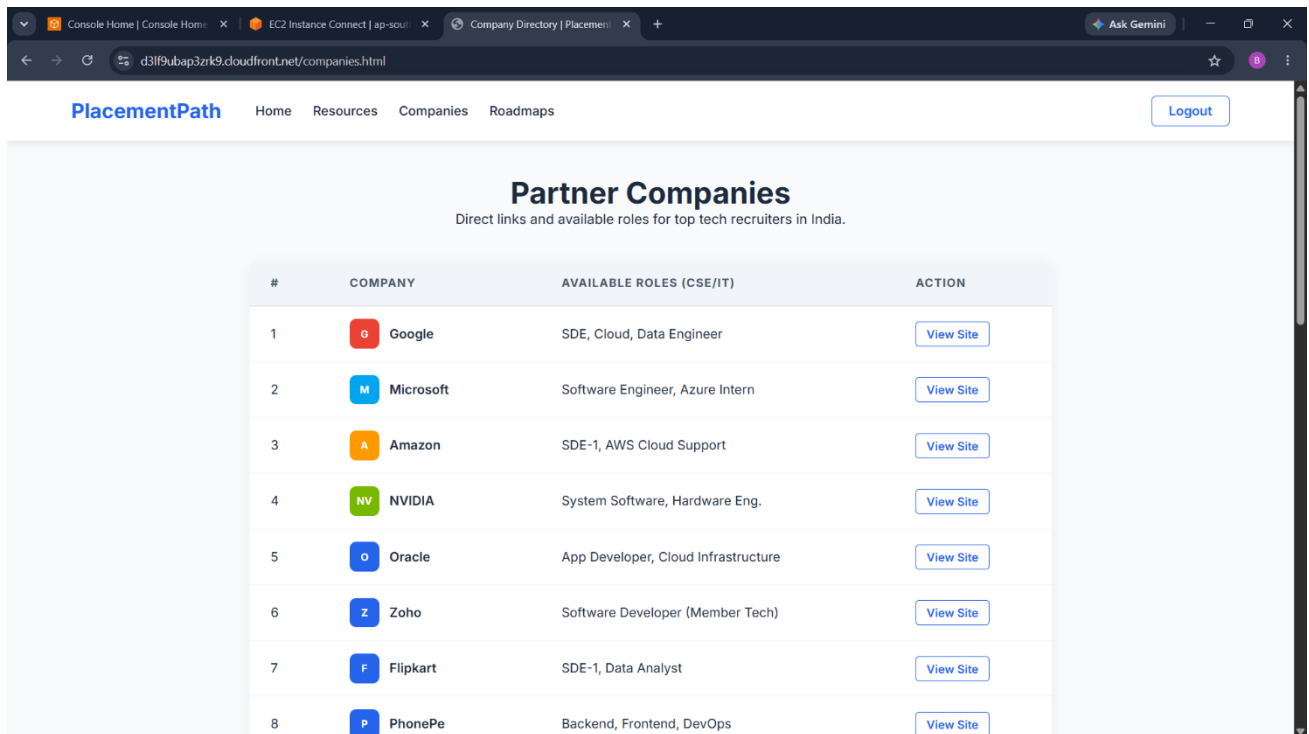


Figure 23. Company Information Page

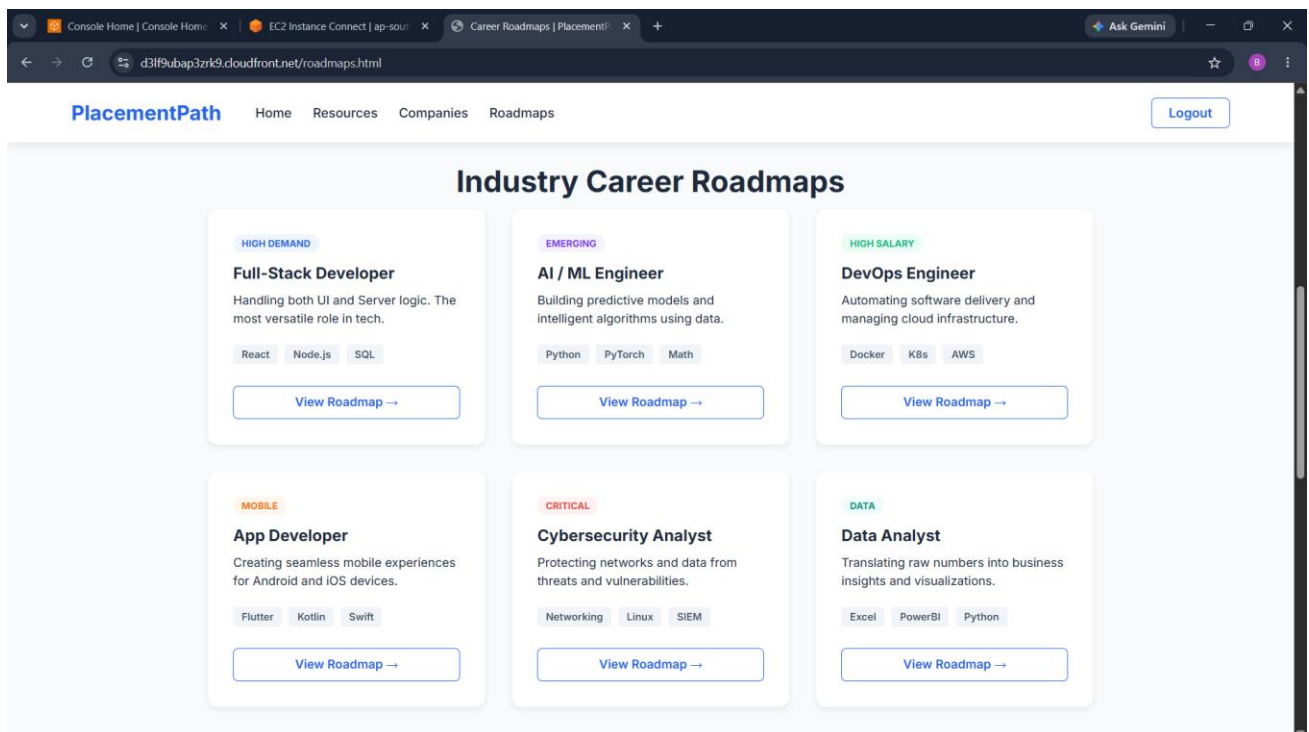


Figure 24. Preparation Roadmaps Page

7. CONCLUSION

The successful development and deployment of **PlacementPath** demonstrate the practical application of cloud computing concepts in building a secure and scalable full stack web application. By integrating modern web technologies with Amazon Web Services (AWS), the project provides a centralized platform where students can access placement-related resources, company information, and preparation roadmaps through a simple and user-friendly interface.

The project successfully utilizes **Amazon S3** for hosting the frontend, **Amazon EC2** for deploying the Flask backend, **Amazon RDS (MySQL)** for managing application data, **Amazon CloudFront** for efficient content delivery, and **AWS IAM** for secure access management. The implementation of **JWT authentication**, password hashing, **Nginx**, and **Gunicorn** further enhances the security, reliability, and performance of the application.

Throughout the development process, valuable practical experience was gained in designing a **three-tier architecture**, deploying cloud-based applications, configuring AWS services, managing databases, and implementing secure authentication mechanisms. The project demonstrates how multiple AWS services can be integrated to build a reliable and efficient cloud-based solution suitable for real-world deployment.

The application successfully met its primary objectives by providing secure user registration, authentication, centralized placement resources, and reliable cloud deployment. The architecture is scalable and can support additional users and features with minimal modifications, making it suitable for future expansion.

The project also provides a strong foundation for future enhancements, such as implementing an administrator dashboard, resume management, job notifications, company recruitment modules, email verification, password recovery, mobile application support, and AI-based placement recommendations. These features can further improve the functionality and user experience of the platform.

In conclusion, **PlacementPath** serves as an effective demonstration of full stack web development integrated with AWS cloud services. The project successfully fulfills the objectives of the **Summer Online Internship – AWS Cloud Computing** by showcasing the practical implementation of cloud technologies, secure web application development, and modern deployment practices in a real-world application.